Build: Infinite Scroll List

Problem: Build a list that loads more items as the user scrolls to the bottom.

What This Tests: Intersection Observer, pagination, useCallback, useRef, loading states.

jsx

JSX Copy

```jsx
function InfiniteScrollList() {
  const [items, setItems] = useState([]);
  const [page, setPage] = useState(1);
  const [loading, setLoading] = useState(false);
  const [hasMore, setHasMore] = useState(true);
  const observerRef = useRef(null);

  const loadMore = useCallback(async () => {
    if (loading || !hasMore) return;
    setLoading(true);

    // Simulate API call
    await new Promise(resolve => setTimeout(resolve, 800));
    const newItems = Array.from({ length: 20 }, (_, i) => ({
      id: (page - 1) * 20 + i + 1,
      title: `Item ${(page - 1) * 20 + i + 1}`,
    }));

    setItems(prev => [...prev, ...newItems]);
    setPage(prev => prev + 1);
    setHasMore(page < 5); // Stop after 5 pages
    setLoading(false);
  }, [page, loading, hasMore]);

  const lastItemRef = useCallback(
```

```
    observerRef.current = new IntersectionObserver(([entry]) => {
      if (entry.isIntersecting && hasMore) {
        loadMore();
      }
    });

    if (node) observerRef.current.observe(node);
  },
  [loading, hasMore, loadMore]
);

useEffect(() => { loadMore(); }, []);

return (
  <div style={{ height: 500, overflow: 'auto' }}>
    {items.map((item, index) => (
      <div
        key={item.id}
        ref={index === items.length - 1 ? lastItemRef : null}
        style={{ padding: 16, borderBottom: '1px solid #eee' }}
      >
        {item.title}
      </div>
    ))}
    {loading && <p style={{ textAlign: 'center', padding: 16 }}>Loading
    {!hasMore && <p style={{ textAlign: 'center', padding: 16 }}>No more
  </div>
);
}
```

Key Concepts: Intersection Observer API, callback ref pattern, pagination state, preventing duplicate fetches with loading guard, end-of-data handling.

QUICK CHECK — no marks, just for you.

1. In the InfiniteScrollList component, loadMore begins with `if (loading || !hasMore) return;`. What is the primary purpose of this early return?

- It disconnects the Intersection Observer so the sentinel element stops being watched
- It prevents duplicate fetches while a request is already in flight and stops fetching once the data is exhausted
- It forces React to re-render the list before the new items are appended to state

Check answer

2. Inside lastItemRef, why is `observerRef.current.disconnect()` called before a new IntersectionObserver is created?

- Because disconnect() is what actually triggers the isIntersecting callback for the new node
- Because IntersectionObserver instances can only be constructed once per component lifetime
- To flush pending state updates from setItems before the next page is requested
- So the observer from the previous render stops watching the old last item, leaving only the current last item observed

Check answer

3. The list renders `ref={index === items.length - 1 ? lastItemRef : null}`. What does attaching the callback ref this way accomplish?

- It gives the last item a stable key so React can reuse its DOM node across pages
- Only the final rendered item acts as the sentinel, so loading is triggered when the user scrolls to the bottom of the list
- Every item is observed, but React ignores all callbacks except the last one to fire

It ensures the last item is measured for height so the scroll container can be sized correctly

Check answer

Previous
Build: Nested Circles (Recursive Component)

Next
Build: Multi-Step Form with Validation

LEARN MORE

ARTICLES & DOCS

- Intersection Observer API (MDN)
  MDN

VIDEOS



- Easiest way to infinite scroll
  YouTube



- Infinite scrolling — machine coding round
  YouTube



- Infinite scroll — full tutorial
  YouTube

Hand-picked references for this lesson.